LINUX / SYSTEMS

Cron & Scheduling Visual Guide

Crontab syntax, systemd timers compared, pitfalls, and logging.

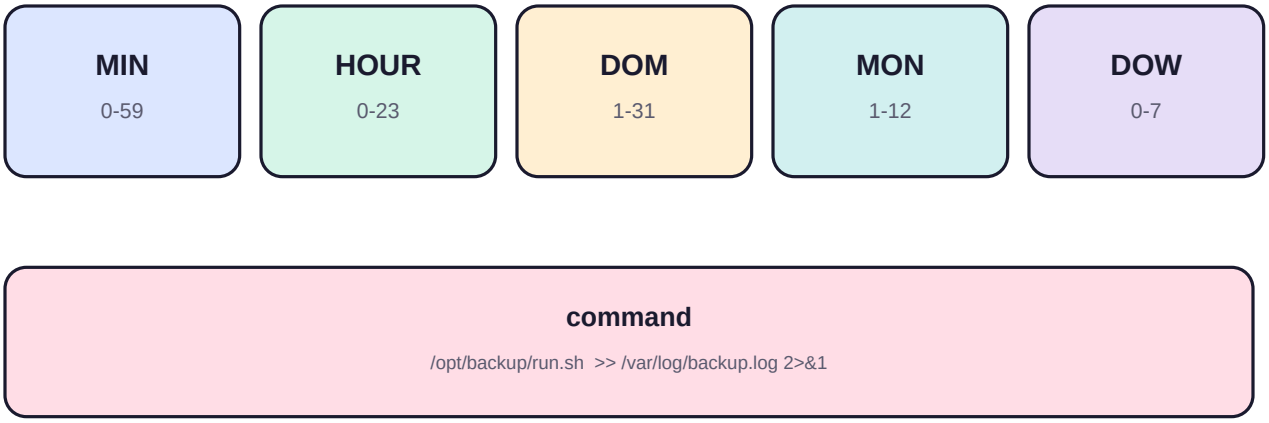
ANIMATED WALKTHROUGH

Step through pages to play the animation

STEP 1/6

Crontab Anatomy: Five Fields

Minute, hour, day-of-month, month, day-of-week — then the command.




```
# m h dom mon dow  command
30 2 * * *   /opt/backup/run.sh      # 02:30 daily
*/15 * * * * /opt/metrics/push.sh    # every 15 min
0 9 * * 1     /opt/reports/weekly.sh # Mondays 09:00
```

DOM and DOW are OR'd, not AND'd: '0 0 1 * 1' runs on the 1st AND every Monday. This surprises everyone once.

STEP 2/6

Special Strings & Step Values

Shorthand for the common rhythms.

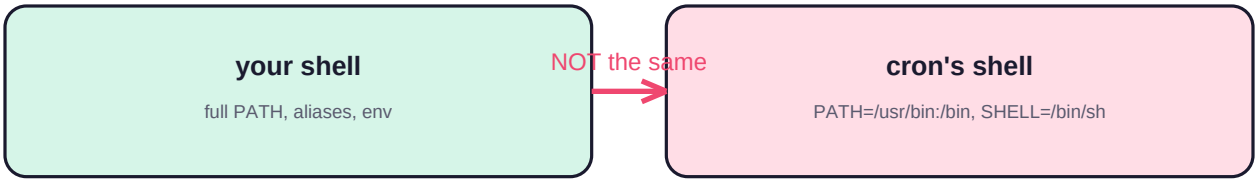
Shorthand	Equals	Use
@reboot	run once at boot	start user agents
@daily	0 0 * * *	midnight jobs
@hourly	0 * * * *	hourly sweeps
*/10 * * * *	every 10 min	pollers
0 2-4 * * *	02:00–04:00 hourly	nightly window
RANDOM*	systemd only	see step 5

```
$ crontab -e      # edit YOUR crontab
$ crontab -l      # list it
$ sudo crontab -e -u appuser   # edit another user's
```

STEP 3/6

The Environment Trap

Cron's environment is nearly empty — your shell's is not.



- 'Works in my shell, fails in cron' is ALWAYS the environment. Use absolute paths everywhere.
- Set what you need at the top of the crontab: PATH=..., SHELL=/bin/bash, MAILTO=you@x.
- A literal % in the command becomes a NEWLINE (stdin to the command) — escape it: \%.
- Source a profile or env file inside the script itself — don't rely on interactive dotfiles.

STEP 4/6

Logging: Cron Is Silent by Default

No output handling = mailed to you (or lost). Capture it yourself.

```
# GOOD: explicit logs, dated, rotated by logrotate
30 2 * * * /opt/backup/run.sh >>/var/log/backup.log 2>&1
# BETTER: also timestamp each run
30 2 * * * /opt/backup/run.sh 2>&1 | ts >>/var/log/backup.log
# BEST for systemd users: a timer unit — output lands in the journal
```

- Always redirect BOTH streams (2>&1) — stderr-only failures are invisible otherwise.
- MAILTO=alerts@example sets where cron mails output; empty MAILTO disables mail.
- Check the cron daemon's own log when jobs never fire: /var/log/cron or journalctl -u cron.

STEP 5/6

Cron vs systemd Timers

Pick by feature need, not fashion.

cron

one-liners · any unix · no deps

systemd timer

deps · journal · catch-up · jitter

Need	Winner
run after network/db is up	timer (After=)
catch up missed runs (laptop asleep)	timer (Persistent=true)
randomized start (thundering herd)	timer (RandomizedDelaySec=)
accuracy to the second	timer (AccuracySec=)
five-minute one-liner	cron
works on macOS/BSD	cron

STEP 6/6

When the Job Never Runs: Checklist

Debug in this order.

- ✓ Is cron running? systemctl status cron (or crond)
- ✓ Syntax valid? paste the line into crontab.guru
- ✓ Absolute paths in the script? (environment trap)
- ✓ Executable bit set? chmod +x the script
- ✓ % escaped? MAILTO set? logs redirected?
- ✓ Systemd timer instead? systemctl list-timers --all

Add a heartbeat line first: '* * * * * date >> /tmp/cron-alive.log' — if that never appears, the problem is cron itself, not your script.

STEP ✓/✓

Key Takeaways

Cron & Scheduling

- Five fields: minute hour day-of-month month day-of-week — DOM/DOW are OR'd.
- Cron's environment is minimal: absolute paths, set PATH/SHELL/MAILTO, escape %.
- Redirect both streams to a log; silent cron jobs are undebuggable cron jobs.
- Timers win on dependencies, catch-up, jitter, and journal logging; cron wins on simplicity.
- Debug checklist: daemon running → syntax → env → perms → escaping → logs.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev